Software Design Specification (SDS)

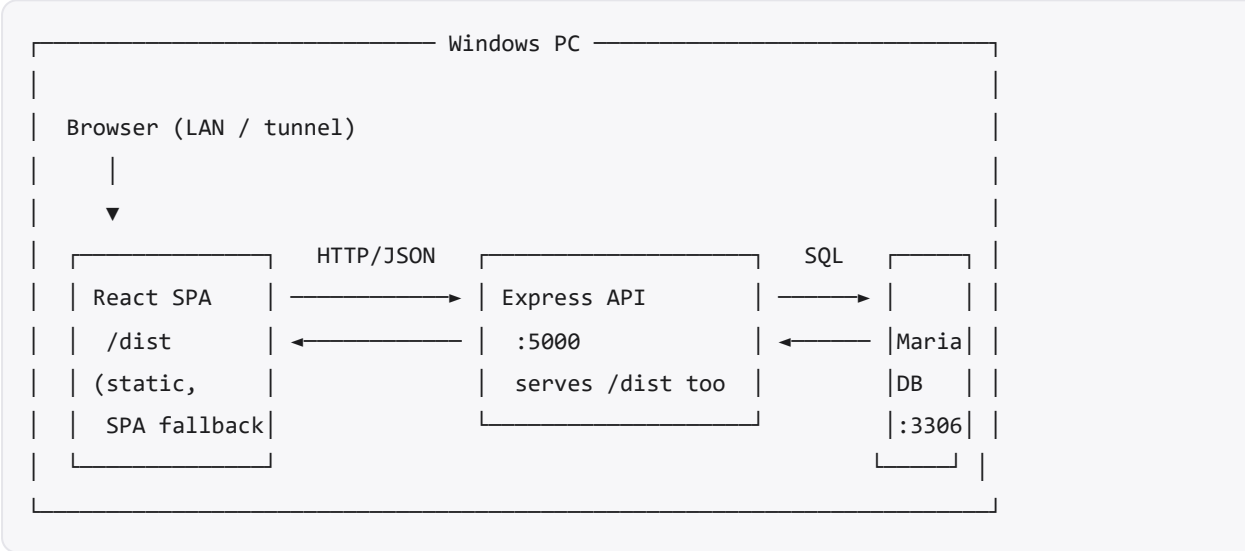
MWINUKA ENTERPRISES CO. LTD — Point of Sale System (MVP)

- **Version:** 1.1
- **Date:** 2026-10-01
- **Status:** Live MVP

---

1. Architecture Overview

Three-tier, single-deployment design on one Windows PC:



- **Monolith serving both API and UI** — Express serves `/api/*` and statically serves the Vite build (`frontend/dist`), with an SPA fallback to `index.html`.
- **Stateless API** (JWT) + **stateful session rows** (revocable).

Tech Stack

Layer	Technology
Frontend	React 18, Vite, Tailwind CSS v4, react-router-dom, lucide-react icons
Backend	Node.js (≥18), Express, jsonwebtoken, bcryptjs, dotenv, mysql2, nodemailer
Database	MariaDB (`mwinuka_pos`)
Build/run	`npm run build` (frontend), `node server.js` (backend)

Directory Layout

```
MWINUKA POS/
├─ backend/
│  ├─ server.js            # app bootstrap, /api guard, static SPA serve
│  ├─ .env                 # PORT, DB credentials, JWT_SECRET
│  ├─ config/
│  │  ├─ db.js             # connection pool + initDb
│  │  └─ seed.js           # DDL, migrations, default users/data
│  │   (.env also holds SMTP_* and ALERT_EMAIL)
│  ├─ notify.js            # Gmail SMTP transport, notifyLogin, sendTestAlert
│  ├─ middleware/auth.js   # auth + adminOnly
│  ├─ config/audit.js      # logAudit, parseModules
│  └─ routes/              # auth, products, orders, customers, reports, settings, audit
└─ frontend/
   └─ src/
      ├─ main.jsx, App.jsx         # entry, routes, gates
      ├─ constants.js              # modules, access helpers, pack/price math
      ├─ context/
      │  ├─ AuthContext.jsx        # JWT/session storage, fetch wrapper, /auth/me refresh
      │  └─ AppContext.jsx         # data loading (products/orders/customers/settings/cart)
      ├─ components/layout/        # Sidebar, Navbar (menu dropdown + live search), Footer
      └─ pages/                    # Login, Home, Products, Orders, CreateOrder, Stock,
                                   # Reports + admin/ (Dashboard, Products, Stock, Sales,
                                   # Orders, Customers, Users, Devices, Reports, Audit, Set
```

---

2. Frontend Design

2.1 Routing (`App.jsx`)

Route	Guard	Layout
`/login`	public (redirects if logged in)	—
`/`, `/products`, `/orders`, `/orders/new`, `/stock`, `/reports`	`ProtectedRoute`	SalesLayout (Navbar + Footer)
`/admin/*` (11 modules)	`AdminLayout` + `AdminGate` per module	Sidebar / bottom tabs

- `ProtectedRoute` → redirects to `/login` when not authenticated.
- `AdminGate` → checks `canAccess(user, module)`; redirects to the first granted module or `/`.

2.2 Authentication Context & Fetch Wrapper (`AuthContext.jsx`)

- **Module-level state** `session = { id, sessionId, token }` read synchronously from `localStorage["mwinuka_auth"]` so the fetch wrapper works before React hydrates.

- **Global `fetch` override** injects `x-user-id`, `x-session-id`, `Authorization: Bearer <token>` headers; on `401` (excluding `/auth/login`, `/auth/logout`) calls `expire()` → clears session → auto-logout.
- `sent === session` guard prevents a stale `401` from wiping a fresh login.
- **Heartbeat:** every 30 s `POST /auth/ping`; alongside it `GET /auth/me` refreshes `admin_modules`/role/name when changed (identity-preserving `setUser`, so lists don't refetch every tick). Also refreshes on window focus/`visibilitychange`.
- `login()` sets the module `session` synchronously, then `setUser`; `logout()` clears both.

2.3 Data Context (`AppContext.jsx`)

Holds `products`, `ordersList`, `customers`, `settings`, `cart`. The load effect is gated on `user` (`if (!user) return`) with `user` in the dependency array so lists load/refresh after login and after permission/profile changes.

2.4 Permissions & Domain Math (`constants.js`)

- `canAccess(u, key) = u.role==='admin' || u.admin_modules.split(',').includes(key)`
- `modulesOf(u)` parses the CSV of granted modules.
- Pack/price helpers: `packPerCarton`, `unitBuyCost`, `piecesForQty`, `packBreakdown`, `stockPack`, `stockParts`, `formatPackBreakdown` — convert cartons/dozens/pieces and validate that a selling price exceeds the per-unit buy cost.

2.5 UI/UX

iOS-style design tokens in `index.css` (`@theme`): primary `#007AFF`, surface `#F2F2F7`, danger `#FF3B30`, success `#34C759`, warning `#FF9500`, system font stack, safe-area helpers (`safe-bottom`, `pb-safe`). Responsive: sidebar hidden below `lg` → iOS bottom tab bar; tables use `overflow-x-auto`; top bar has a menu dropdown (granted Admin modules + Sales links) and a live product search.

2.6 Receipt Printing (`components/InvoicePrint.jsx`)

80 mm printable layout (`index.css` classes), themed print stylesheet (`@media print`): hides the app, shows only `.print-area`, black header/table, zebra rows, boxed BALANCE block. Daily reports use `.report` / `.report-table` print styles.

---

3. Backend Design

3.1 Bootstrap (`server.js`)

1. `dotenv` config, `cors()`, `express.json()`.
2. **API guard:** everything under `/api` runs `auth` middleware **except** `PUBLIC_ROUTES = [POST /auth/login, GET /health]`.
3. `initDb()` then `seed()` (idempotent DDL + migrations + default users).
4. Mount routers: `/api/auth`, `/products`, `/orders`, `/reports`, `/customers`, `/settings`, `/audit`.

5. `express.static(DIST)` + regex SPA fallback → `index.html`.
6. Listen on `PORT` (default 5000).

3.2 Middleware (`middleware/auth.js`)

- **`auth`:** parse `Bearer` token → `jwt.verify` → **re-read user from DB** (`SELECT … FROM users WHERE id=? AND active=1`) → require `x-session-id` and validate an open `sessions` row → throttled `last_seen` update → set `req.user`.
- **`adminOnly`:** `req.user.role === 'admin'` else `403`.

  The per-request DB read makes permission grants effective immediately (server-authoritative).

3.3 Authentication/Users (`routes/auth.js`)

Endpoint	Access	Purpose
`POST /auth/login`	public	Verify credentials → JWT (24 h) + insert `sessions` row → return `{token, session_id, user}`
`POST /auth/logout`	auth	Set `logout_at` on the session
`GET /auth/me`	auth	Return current user (name, role, username, `admin_modules`, `allow_manual_price`) fresh from DB
`POST /auth/ping`	auth	Touch `last_seen`
`GET /auth/sessions`	admin	All sessions + online/active/total counts
`POST /auth/sessions/:id/revoke`	admin	Force sign-out (sets `logout_at`); audit
`GET /auth/users`	admin	List users
`POST /auth/users`	admin	Create user (role, `admin_modules`, `allow_manual_price`)
`PUT /auth/users/:id`	auth	Admin: any field. Non-admin: own name/username/email only (role/active/modules refused)
`POST /auth/change-password`	auth	Own password; admin may pass `userId`
`DELETE /auth/users/:id`	admin	Remove user

On a successful login the route calls `notifyLogin(user, session)` **after** the session row is inserted and **before** the response is sent. The call is fire-and-forget (`.catch()`) so SMTP latency or failure can never delay or reject a valid login.

3.4 Business Modules (route pattern)

- **Products** — CRUD + per-unit price matrix in `product_prices`; pack factors; stock/deactivate; admin-only create/update/delete.
- **Orders** — create order (with items, invoice number), list/filter, detail, pay-items, update items/status, delete; admin gates on mutation where required.
- **Customers** — CRUD, balances; admin gates on edit/delete.
- **Settings** — GET public, PUT admin-only. Accepts the business profile plus `login_alert_enabled` / `login_alert_email`. Exposes `POST /settings/test-alert` (admin) which calls `sendTestAlert()` so delivery can be verified on demand.
- **Reports** — sales summary (revenue = paid items), reset-orders / reset-all (admin).
- **Audit** — filterable action log (admin).

3.5 Database Schema (MariaDB, `mwinuka_pos`)

```
users          id, name, username(UNIQUE), email(UNIQUE), password(bcry
               role('admin'|'sales_staff'), active, admin_modules(CSV),
               created_at
products       id, name, category, price_wholesale, price_retail, price
               stock, min_stock(20), unit, active, buy_price,
               pieces_per_carton, dozens_per_carton, pieces_per_dozen(1
product_prices id, product_id(FK), unit, retail_price, wholesale_price,
               UNIQUE(product_id, unit)            -- active price matr
customers      id, name, phone, email, created_at
orders         id, order_no(UNIQUE), customer_id(FK), salesman_id(FK),
               payment_method, notes, status, order_date, created_at
order_items    id, order_id(FK cascade), product_id(FK), quantity, unit
               unit_price, total, paid(TINYINT)
settings       id, setting_key(UNIQUE), setting_value
                -- keys include business_name, business_phone, business
                --          business_address, business_tagline,
                --          login_alert_enabled, login_alert_email
audit_logs     id, user_id, username, role, action, details, created_at
sessions       id, user_id(FK), device_label, browser, os, ip, login_at
```

`seed()` additionally performs idempotent `ensureColumn` migrations, legacy price-column → matrix migrations, and marks `order_items.paid=1` for completed orders.

3.6 Auth Flow (Sequence)



Revocation: `logout_at` set → subsequent requests fail `auth` → fetch wrapper `expire()` → redirect to `/login`.

3.7 Login Notification Module (`notify.js`)

A self-contained module with no route dependencies:

Export	Behaviour
`smtpConfigured()`	True when both `SMTP_USER` and `SMTP_PASS` are present in `.env`
`notifyLogin(user, session)`	Builds the subject/body (name, username, role, device, browser, OS, IP, timestamp), resolves the recipient (`login_alert_email` setting → `ALERT_EMAIL` fallback), sends via `notifyLogin(user, session).catch(log)`
`sendTestAlert(req)`	Sends a diagnostic message to the configured recipient; rejects with a clear message when SMTP is unconfigured

- **Transport:** `nodemailer` SMTP against `SMTP_HOST:SMTP_PORT` (Gmail `smtp.gmail.com:587`, STARTTLS) with `SMTP_USER` / `SMTP_PASS`. `secure` is derived from the port, so port 465 works without extra config.
- **Sender:** `SMTP_FROM`, falling back to the SMTP user itself.
- **Config precedence:** recipient = DB setting `login_alert_email` → env `ALERT_EMAIL` → SMTP user.
- **Failure isolation:** `notifyLogin()` is invoked with `.catch()`; any SMTP failure is logged to the server log and discarded. Login success is never contingent on email delivery.
- **Secrets:** `.env` holds `SMTP_PASS` (a 16-char Google app password, never the account password). It is not committed, not returned by any endpoint, and has no UI field.

Frontend: `AdminSettings.jsx` renders a *Login Alerts* card with an enable toggle, recipient input, *Save* and *Send Test Email*. Only the toggle/email are editable — the UI states that credentials are server-side.

---

4. Security Model

- **Passwords:** bcrypt (cost 10); never stored/returned in responses.
- **Tokens:** JWT HS (64-hex `JWT_SECRET` in `.env`), 24 h expiry, no permissions inside the payload.
- **Authorisation:** server re-queries the user row each request; role + `admin_modules` decide route access; UI merely hides what the server enforces.
- **Sessions:** revocable; force sign-out is immediate; `last_seen` throttled DB writes.
- **Headers:** auth via the wrapper so every fetch is protected; `CORS` enabled for LAN use.
- **Notification credentials:** the Gmail app password lives only in `backend/.env`. No endpoint echoes it, the Settings UI never renders it, and `test-alert` returns only a success/failure message — never the credential or the SMTP error body.
- **Owner alerts:** every login is emailed to the configured recipient, giving the shop owner an out-of-band record of who accessed the POS and from where.

---

5. Deployment & Operations

1. `cd frontend && npm run build` → produces `frontend/dist`.
2. `cd backend && node server.js` (or `npm start`), MariaDB running on `:3306`.
3. Access `http://192.168.1.116:5000` on LAN.
4. Off-site: ngrok (`ngrok http 5000 --url=<static-domain>`) or Cloudflare Quick Tunnel, run from a terminal that stays open.
5. Restart note: mysqld is launched manually via `C:\xampp\mysql\bin\mysqld.exe`; the backend serves the latest `dist` automatically on rebuild.
6. `.env` **changes require a backend restart** — env vars are read once at boot, so SMTP credentials only take effect after `node server.js` is restarted.

5.1 Gmail Alert Setup (verified working)

1. Gmail account must have **2-Step Verification** enabled.
2. Create a 16-character app password at `myaccount.google.com/apppasswords`.

3. Add to `backend/.env` (no spaces in the app password):

```
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=<gmail address>
SMTP_PASS=<16-char app password>
SMTP_FROM="Mwinuka Enterprises Co Ltd <gmail address>"
ALERT_EMAIL=<default recipient>
```

1. Restart the backend, then Admin → Settings → *Send Test Email*.

Signups for these services must be completed on a network without TLS inspection (e.g. phone hotspot) — the shop network blocks them.

---

6. Known Constraints / Notes

- Non-elevated CLI cannot configure static IP or system services; admin PowerShell used for network config.
- GitHub/HTTPS downloads are throttled on the shop network (Fortinet inspection); signups/tunnels may need a different network (phone hotspot).
- The `sales_staff` account `john@mwinuka.co.tz` had its password changed externally; credentials are not stored in the repo.
- Login alerts depend on outbound `smtp.gmail.com:587`. TLS-inspecting firewalls (Fortinet) may block it; LAN operation is unaffected either way. SMS alerts are not implemented and would need a third-party gateway account.
- A Vercel deployment was attempted and is not viable for this architecture — serverless functions cannot host the long-running Express server or reach the local MariaDB instance, and it returned `FUNCTION_INVOCATION_FAILED`. Off-site access is provided by a local tunnel instead.
- Credential hygiene: a real Gmail password was once pasted into a chat session. It should be rotated, and app passwords revoked/rotated after any credential exposure.

---

7. Revision History

Version	Date	Change
1.0	2026-09-30	Initial MVP design baseline.
1.1	2026-10-01	Added the login-notification module (`notify.js`), settings keys and test-alert endpoint, SMTP environment contract, and deployment notes.